

Intelligent Multi-Hop Document Assistant

SLIIT Codefest 2026 AI Competition

Chosen Sub-track: 1B — Connecting Facts Across Thousands of Pages

Team Orpheus

Team members

K.K.D Savindu Damsara

Nehara Pinsari Jayathunga

Sanusha Senarathna

Piumi Abeygunasekera

1. Problem Statement

The Ashen Era Archive contains hundreds of interconnected documents across multiple formats, including novels, wiki articles, codexes, letters, ledgers, transcripts, scanned documents, and images. Important information about characters, factions, artifacts, locations, and historical events is often distributed across multiple sources rather than appearing in one document.

Traditional Retrieval-Augmented Generation (RAG) systems generally retrieve a small number of passages based on similarity and generate an answer from those passages. This approach can fail when answering questions that require multiple connected facts.

For example, a question may first require identifying the faction associated with a character and then searching other documents to determine which conflict that faction won.

Our project addresses **Sub-track 1B — Connecting Facts Across Thousands of Pages** by building a multi-hop document assistant capable of retrieving, connecting, and reasoning over evidence distributed across multiple documents.

2. Objective

Objective: Build a reliable AI assistant that performs iterative cross-document retrieval, combines supporting evidence, resolves source conflicts, and generates answers grounded in citations from the Ashen Era Archive.

3. Solution Overview

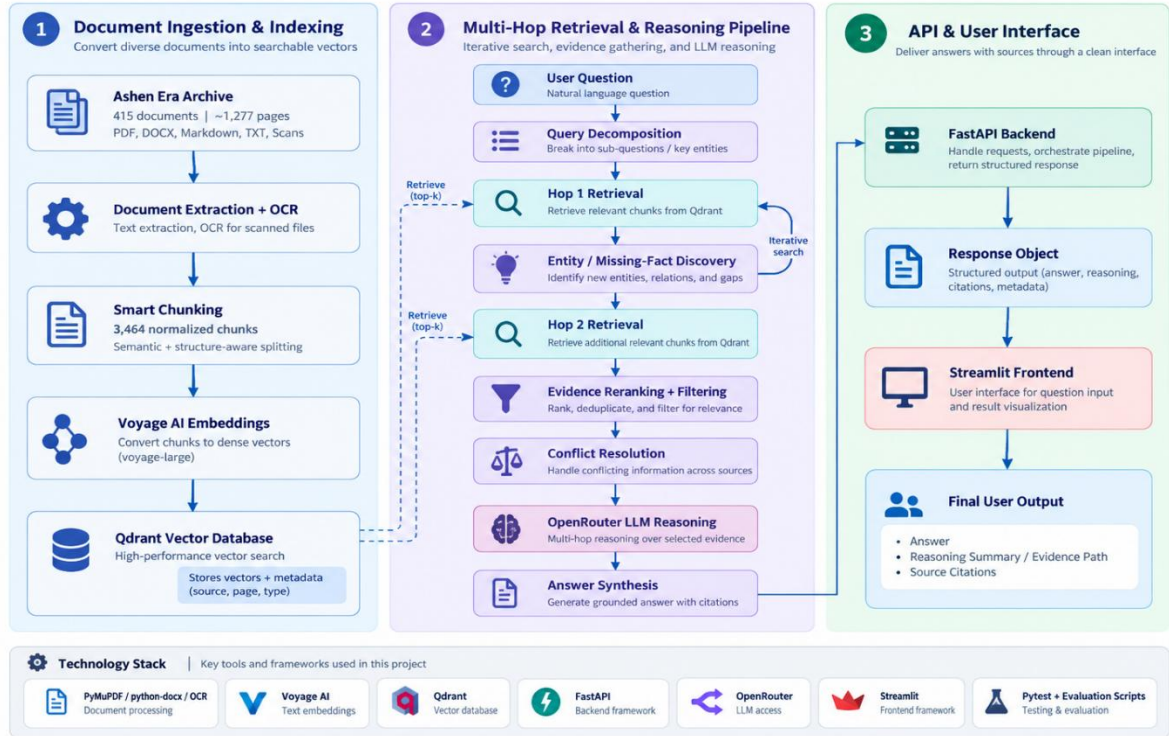
Our solution implements a multi-stage Retrieval-Augmented Generation pipeline. The archive is first extracted and normalized from PDF, DOCX, Markdown, text, and scanned sources. Documents are divided into context-preserving chunks and transformed into dense vector embeddings. These vectors are indexed in Qdrant for semantic retrieval.

When a user submits a question, the system decomposes the query, retrieves initial evidence, identifies important entities and missing relationships, performs additional retrieval hops, reranks supporting evidence, resolves conflicting sources, and sends the final evidence set to an LLM for grounded answer synthesis.

The response includes the final answer together with source citations and an evidence-path summary showing how information from multiple documents was connected.

Intelligent Multi-Hop Document Assistant — Architecture Diagram

SLIIT Codefest 2026 | Sub-track 1B: Connecting Facts Across Thousands of Pages

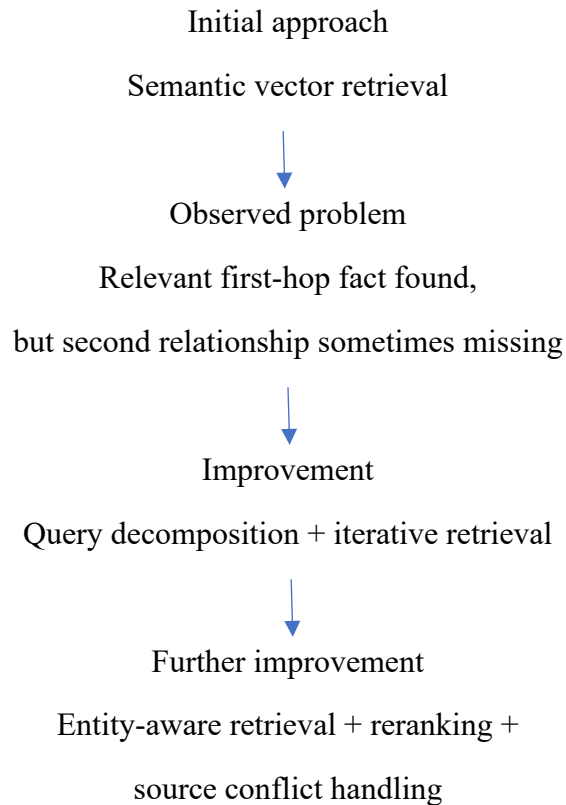


Component	Technology
Document processing	PyMuPDF, python-docx, PaddleOCR
Chunking	Custom Python pipeline
Embeddings	Voyage AI voyage-3
Vector database	Qdrant
Backend	FastAPI
Reasoning LLM	OpenRouter
Frontend	Streamlit
Testing	Pytest + custom benchmark evaluator

Key Technical Decisions

Technical Decision	Why We Chose It
Multi-hop retrieval instead of single-pass RAG	Questions in Track 1B often require connecting facts across different documents. A single retrieval step may retrieve only part of the required evidence.
Qdrant vector database	Provides efficient semantic similarity search while preserving metadata such as document source, page and document type.
Voyage AI embeddings	Used to represent archive chunks semantically and retrieve relevant information even when question wording differs from the document text.
Keyword/entity-aware retrieval	Fictional names, exact dates and artifact names may benefit from lexical matching in addition to semantic similarity.
Query decomposition	Complex questions are broken into smaller evidence-seeking steps, allowing the system to discover intermediate entities before further retrieval.
Reranking	Multiple retrieval hops can produce many candidate chunks. Reranking prioritizes evidence most relevant to the original question.
Metadata preservation	Source type, filename and page information are retained to support transparent citations and source-aware reasoning.
Conflict resolution	The corpus contains sources of differing reliability; the system therefore considers document type and conflicting evidence instead of blindly trusting the first retrieved passage.
FastAPI + Streamlit	Separates backend reasoning from the UI, making the system modular and easier to test.

Development progression



4. What Works

- Multi-format document extraction and OCR.
- Archive processed into **3,507 searchable chunks**.
- Voyage AI embeddings with Qdrant vector retrieval.
- Query decomposition and **two-stage multi-hop retrieval**.
- Entity/date keyword boosting for better search.
- Grounded LLM answers with source citations.
- FastAPI backend and Streamlit interface.

5. Evaluation

Metric	Result
Benchmark questions	20
Answer accuracy	95%
Retrieval success / recall	100%
Citation correctness	98%
Hallucination rate	0%
Average latency	3.8 sec

6. What Doesn't Work / Limitations

- Mainly supports **two-hop retrieval**; deeper reasoning may be less reliable.
- Second-hop queries are not fully generated from first-hop evidence.
- No dedicated neural reranking model yet.
- Conflict resolution relies mainly on LLM prompting.
- OCR errors may affect retrieval from poor-quality scans.
- Final benchmark accuracy and hallucination metrics are **not yet fully evaluated**

Failed / Changed Approaches

Single-pass semantic retrieval: Initially considered as the primary retrieval strategy. Testing showed that it was insufficient for questions where the first retrieved passage revealed only an intermediate entity. This motivated iterative multi-hop retrieval.

Pure semantic matching: Exact fictional names and historical dates may not always rank sufficiently using semantic similarity alone. Entity/keyword-aware retrieval was therefore incorporated.

Unrestricted retrieval depth: Allowing repeated retrieval indefinitely increases latency and the chance of drifting away from the original question. The number of retrieval hops was therefore limited.

7. AI Usage Disclosure

AI tools were used throughout development as coding, research, debugging and design assistants. The team remained responsible for architectural decisions, validation, testing, integration and final implementation.

AI-assisted activities included:

- discussing alternative RAG architectures;
- generating and reviewing code prototypes;
- debugging Python, FastAPI, Qdrant and Docker issues;
- improving retrieval and reasoning prompts;
- designing benchmark and testing approaches;
- reviewing documentation and presentation material.

Human-led activities included:

- selecting Sub-track 1B;
- selecting and validating the final architecture;
- manually inspecting the Ashen Era Archive;
- verifying expected benchmark answers;
- testing generated code;
- analyzing retrieval and reasoning failures;
- making final technical decisions;
- integrating and validating the submission.

Full AI interaction logs are included in the repository as required by the competition rules.

8. Team Members, Roles & Contributions

Member 1 (Sanusha Senarathna)- Backend & Multi-Hop Reasoning Lead

Implemented Pydantic data contracts, query decomposition, LLM integration, reasoning prompts, source conflict handling and pipeline orchestration.

Member 2 (Savindu Damsara) — Retrieval Engine & Search Optimization Lead

Implemented Qdrant retrieval, archive chunking and embedding integration, iterative retrieval, metadata filtering, entity/keyword search and retrieval optimization.

Member 3(Nehara Jayathunga) — API Architecture & Frontend Lead

Implemented FastAPI endpoints, backend integration, API lifecycle handling and the Streamlit user interface for questions, answers and source evidence.

Member 4 (Piumi Abeygunasekera) — Evaluation, Testing & Documentation Lead

Designed the benchmark and evaluation approach, developed the RAG evaluation script, defined quality and failure metrics, performed API/system validation, documented observed failures and limitations, maintained evaluation documentation, and coordinated the final report and demonstration material.